CHAPTER 1

DESIGNING ESPNCRICINFO

1 Getting Ready: The ESPNcricinfo System 2
2 Requirements for ESPNcricinfo . . . 3
3 Use Case Diagram for ESPNcricinfo 4
4 Class Diagram for ESPNcricinfo . . . 6
5 Sequence Diagram for ESPNcricinfo 17
6 Activity Diagram for ESPNcricinfo . 18
7 Code for ESPNcricinfo 19
8 Summary: Designing ESPNcricinfo . 28
9 Key Insights and Tips: Designing ESPNcricinfo 31

This chapter covers the design of the ESPNcricinfo system, including requirements, use case, class, sequence, activity diagrams, and implementation code.

1 Getting Ready: The ESPNcricinfo System

1.1 Problem definition

ESPNcricinfo is a comprehensive cricket management and broadcasting system that offers real-time and historical data for cricket matches, players, teams, and tournaments. It supports various formats, including Test, ODI, and T20 matches. The platform is designed for multiple stakeholders, admins, coaches, umpires, and commentators, each with distinct roles and responsibilities.
The system enables ball-by-ball tracking of match events, including wickets, scores, and live commentary, providing an engaging experience for viewers and analysts. The admin can add or update all statistics related to players, teams, and matches, ensuring data accuracy and completeness.
ESPNcricinfo also handles tournament management, allowing the admin to add and modify tournaments, assign stadiums and match personnel, and manage team participation. For each tournament, coaches can select a squad of players to represent the team. From this squad, the playing eleven can be chosen for each match.
Additionally, the system maintains points tables, match results, and complete historical data for previous tournaments and games. It includes features for publishing cricket news and updates, managed by the admin or coaches, and provides tools for commentators to input or update commentary during live matches.

1.2 Expectations from the interviewee

ESPNcricinfo is a role-based cricket management system that supports match operations, tournament setup, and the handling of real-time data. During the interview, you are expected to clearly understand the system's actors, their responsibilities, and the use cases that drive the system. You should ask relevant questions that help clarify the system's functional and operational behavior.

Player, team, and match statistics

The system must accurately track player, team, and match statistics.

- How are player statistics updated—manually by the admin or through automation?
- What statistical detail is required (e.g., runs, wickets, strike rate)?
- Should stats be updated after every ball, inning, or match?

Match formats and match management

The system must support all three formats: Test, ODI, and T20.

- Are there format-specific rules that need to be encoded in the system?
- Should match duration or innings logic be handled differently based on format?

Squad submission for tournaments

Each team must submit a tournament squad in advance.

- Who is responsible for submitting the squad?
- Can a coach modify the squad after it has been submitted, and under what conditions?

Admin operations

The admin oversees the creation and management of all entities in the system.

- What validations are needed when adding matches, players, or teams?
- How are stadiums and match personnel assigned?
- Is there a review or approval step before updates go live?

1.3 Design approach

We'll design Cricinfo using the bottom-up approach. For this purpose, we will follow the steps below:

- Identify and design the smallest components, like a ball and a run.
- Use these small components to design larger components, such as an over, a team, and an umpire.
- Repeat the steps above until we design the complete ESPNcricinfo platform.

1.4 Design pattern

During an interview, discussing the design patterns that ESPNcricinfo falls under is always a good practice. Stating the design patterns gives the interviewer a positive impression and shows that the interviewee is well-versed in the advanced concepts of object-oriented design.
Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking, "Define design patterns."

Question:
Which design pattern(s) should be used to design the Cricinfo system? Please elaborate on your choice(s).
Let's explore the requirements for ESPNcricinfo in the next lesson.

1.5

2 Requirements for ESPNcricinfo

In this lesson, we'll list ESPNcricinfo's requirements. This is a crucial step since requirements define the scope of a problem. Therefore, accurately obtaining and thoroughly understanding them from the interviewer will make the design of the rest of the system smooth and easy.
We'll use the notational convention to identify each requirement with a unique label, "Rn," where "R" is short for requirement and "n" is a natural number.

2.1 Requirements collection

The requirements for ESPNcricinfo are defined below:

R1: The system shall allow the admin to add or update players, teams, and match statistics.
R2: The system shall enable the commentator to add or modify ball-by-ball commentary, and allow tracking of every score or wicket per ball.
R3: The system shall support tracking all matches, including Test, ODI, and T20, added or modified by the admin.
R4: The system shall maintain a record of ongoing and historical tournaments and display points tables for all participating teams. The admin is responsible for adding or updating tournaments.

R5: The system shall display results for all previous matches, including televised ones, by utilizing match data added by the admin.
R6: The system will allow the coach to submit a tournament squad—a group of selected players eligible to participate.
R7: The system shall support playing 11 selections from the tournament squad for each match.
R8: The admin shall be able to manage the entire system by adding or modifying tournaments, matches, teams, players, stadiums, umpires, commentators, and updating stats and news.

We've identified our requirements for the problem. In the next lesson, we will define different use cases of ESPNcricinfo.

3 Use Case Diagram for ESPNcricinfo

Let's build the use case diagram of ESPNcricinfo and understand the relationship between its different components.
First, we will define the different elements of our ESPNcricinfo system, followed by the complete use case diagram.

3.1 System

Our system is ESPNcricinfo.

3.2 Actors

Now, we'll define the main actors of ESPNcricinfo.

Primary actors

- **Admin:** The admin is responsible for performing various operations, including adding or modifying tournaments, players, squads, matches, assigning stadiums and personnel, and updating statistics.

- **Coach:** The coach manages players, news, and squad submissions for the team in tournaments.
- **Umpire:** The umpire gets assigned to matches and oversees match-related decisions (assignment only modeled).
- **Commentator:** This actor can add commentary to the match or modify it.

Secondary actors

There is no secondary actor in the system.

3.3 Use cases

In this section, we'll define the use cases for ESPNcricinfo. We have listed the use cases according to their respective interactions with a particular actor.

> **Note:** You may see some use cases repeated multiple times because they are shared among different actors in the system.

Admin

- **Add/modify team:** To add or update a team and its attributes.
- **Add/modify player:** To add a player to the team or update a player's details.
- **Add/modify tournament:** To add a tournament or update its structure.
- **Add/modify match:** To add a new match to the system or update existing details.
- **Assign stadium:** To assign a stadium to a specific match.
- **Assign umpire:** To assign umpires to a match.
- **Assign commentator:** To assign a commentator to a match.
- **Add/update stats:** To update player, match, or team stats in the system.
- **Add/update news:** To publish team or match-related news.
- **Maintain points table:** To keep track of the scores and standings of the matches.

3.4 Coach

- **Add player to team:** To add a player to the team's roster.
- **Submit squad for tournament:** To add a selected squad of players to a tournament.
- **Add/update team news:** To publish news articles or updates for a team.

Umpire

- **View assigned match:** To let umpires see their upcoming assigned matches.

Commentator

- **Add/modify commentary:** To add or edit commentary for a specific ball during the match.

3.5 Relationships

We describe the relationships between and among actors and their use cases in this section.

Generalization

- Add/update stats use case has a generalization relationship with:
  - Add/update player stats
  - Add/update match stats
  - Add/update team stats
- Add match use case has a generalization relationship with:
  - Add T20 match
  - Add an ODI match
  - Add Test match

Associations

The table below illustrates the association between actors and their corresponding use cases.

Admin	Commentator	Coach	Umpire
Add /modify team	Add/modify commentary	Add player to team	View assigned match
Add /modify player		Submit squad for tournament	
Add /modify tournament		Add/update team news	
Add /modify match			
Assign stadium			
Assign umpire			
Assign commentator			
Add /update stats			
Add /update news			
Maintain points table			

Include

- Add/modify match use case has an include relationship with:
  - Add/modify run
  - Add/modify wicket

3.6 Use case diagram

Here's the use case diagram of ESPNcricinfo:

In the next lesson, we'll discuss the class diagram with a detailed explanation of all classes and their relationship with each other.

4 Class Diagram for ESPNcricinfo

In this lesson, we'll identify and design classes, abstract classes, and interfaces based on the requirements gathered from the interviewer in our ESPNcricinfo system.

4.1 Components of ESPNcricinfo

As mentioned, we will design the ESPNcricinfo system using a bottom-up approach.

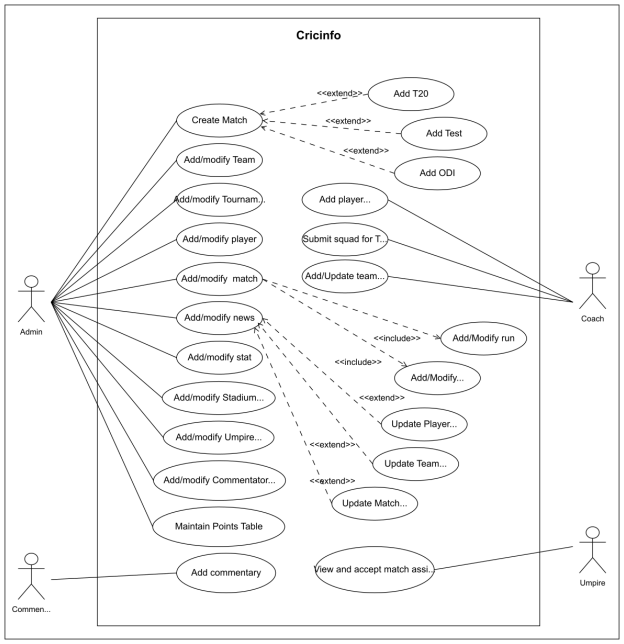


Figure 1.1: The use case diagram of ESPNcricinfo

Admin

The `Admin` class manages the system, adding and modifying updates. The representation of the class is shown below:

R1: The system shall allow the admin to add or update players, teams, tournaments, and match statistics.

Run, ball, and wicket

The `Run` class records the number and type of runs scored on a ball. The `Ball` class records every detail of a ball, such as the number of runs scored and whether it was a wicket-taking ball. The `Wicket` class records the details of the wicket, including its type, the bowler, and the batsman who was declared out.
The mentioned classes are shown below:

R2: The system shall enable the commentator to add or modify ball-by-ball commentary, and allow tracking of every score or wicket per ball.

Over and innings

The `Over` class represents all the details of an over of the innings. The `Innings` class represents the details of a match innings. The two classes are shown below:

Match

The `Match` class is an abstract class that has three child classes, representing the types of matches that can occur.

- The `Test` class
- The `ODI` class
- The `T20` class

The class diagram is shown below:

R3: The system shall support tracking all matches, including Test, ODI, and T20, added or modified by the admin.

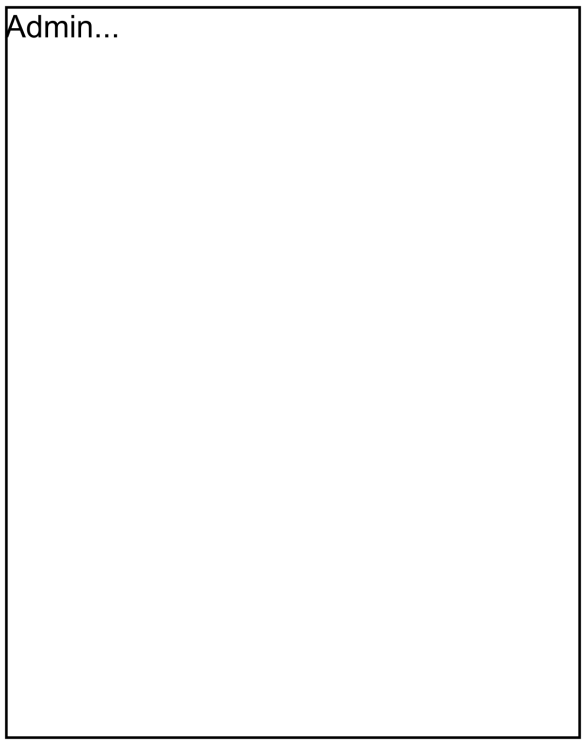


Figure 1.2: The Admin class

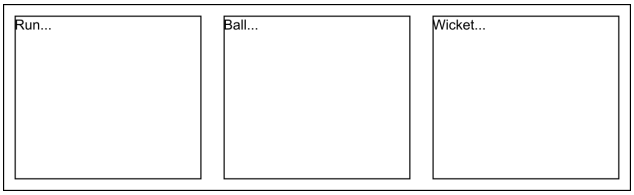


Figure 1.3: The Run, Ball, and Wicket classes

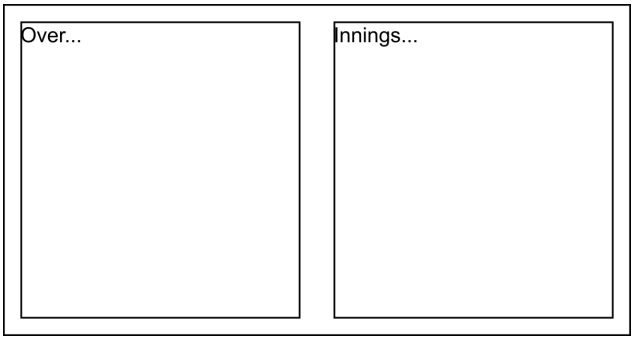


Figure 1.4: The Over and Innings classes

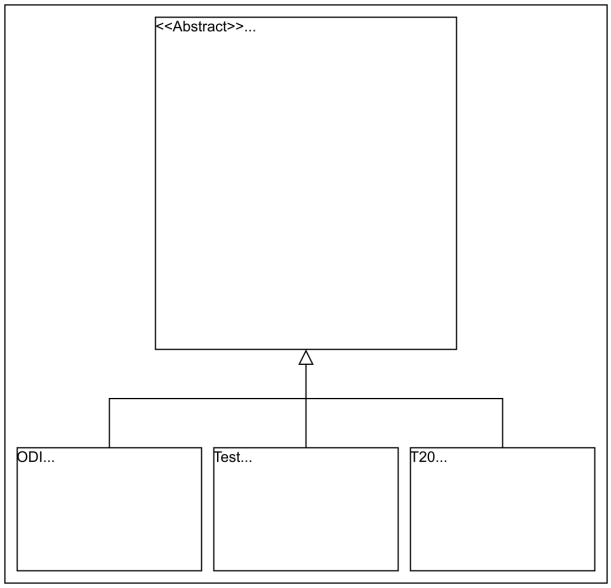


Figure 1.5: The Match and its derived classes

Stadium

The `Stadium` class represents the information about a stadium, including its name, address, and capacity. The UML representation of this class is given below:

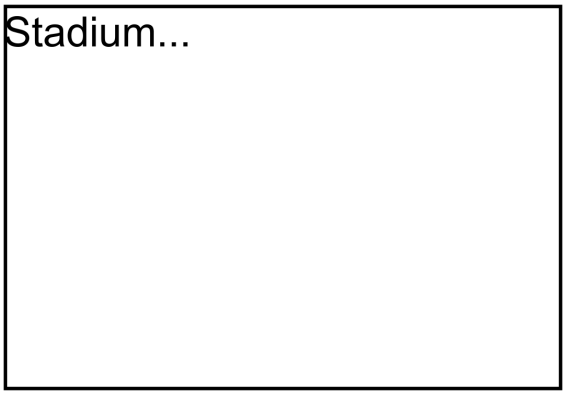


Figure 1.6: The Stadium class

Player, coach, and umpire

The `Player` class includes the information of a player and their statistics. The `Coach` class contains the information of a coach. The `Umpire` class contains the information of an umpire. The three classes are shown below:

R6: The system will allow the coach to submit a tournament squad—a group of selected players eligible to participate.

R8: The admin shall be able to manage the entire system by adding or modifying tournaments, matches, teams, players, stadiums, umpires, commentators, and updating stats and news.

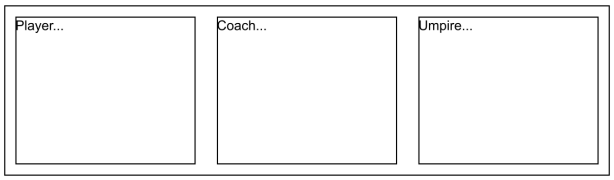


Figure 1.7: The Player, Coach, and Umpire classes

Team, tournament squad, and playing eleven

The `Team` class represents the information about a cricket team, including the list of players, the team coach, and any news related to the team.
The `TournamentSquad` class represents the team members participating in a tournament. The `Playing11` class represents the squad members playing in a match.
The class diagram for these classes is given below:

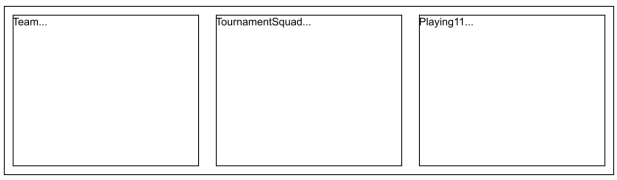


Figure 1.8: The Team, TournamentSquad, and Playing11 classes

R7: The system shall support playing eleven selections from the tournament squad for each match. Although not directly modeled as a use case, this process is implicit in match setup and should be facilitated by the admin or coach through data entry.

Tournament and points table

The `Tournament` class contains information about a cricket tournament. The `PointsTable` class displays the accumulated points and match results of the teams participating in the tournament. These classes are shown below:

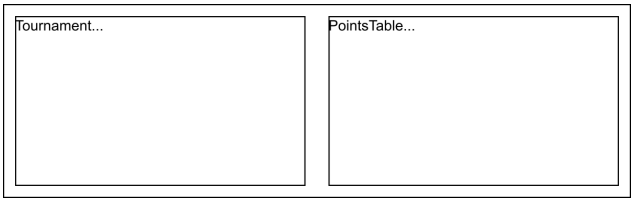


Figure 1.9: The Tournament and PointsTable classes

R4: The system shall maintain a record of ongoing and historical tournaments and display points tables for all participating teams. The admin is responsible for adding or updating tournaments.

Stats

The `Stat` class is an abstract class that extends to `PlayerStat`, `TeamStat`, and `MatchStat` classes. These classes contain important statistics. The UML representation is shown below:

R8: The admin shall be able to manage the entire system by adding or modifying tournaments, matches, teams, players, stadiums, umpires, commentators, and updating stats and news.

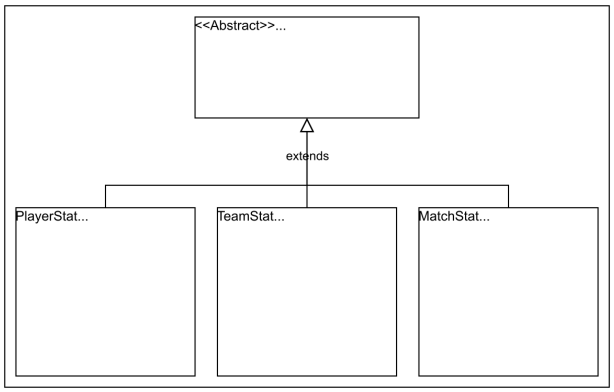


Figure 1.10: Stat and its derived classes

Commentator and commentary

The `Commentator` class records the information about the commentator. The `Commentary` class contains information about the commentary for every ball of an over. The two classes are shown below:

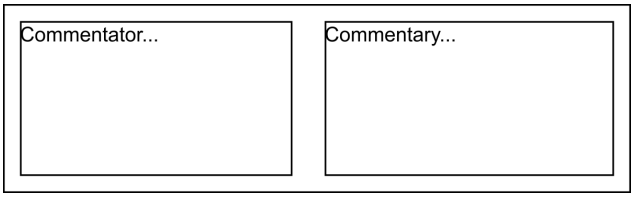


Figure 1.11: The Commentator and Commentary classes

R2: The system shall enable the commentator to add or modify ball-by-ball commentary, and allow tracking of every score or wicket per ball.

News

The `News` class holds the news updates of a team. The definition of this class is given below:

Figure 1.12: The News class

Enumerations

The enumerations required in the ESPNcricinfo system are listed below:

- `MatchResult`: This records the result of a match—a win, loss, canceled, or draw.
- `UmpireType`: This records the type of umpire—field umpire, third umpire, or reserved.
- `WicketType`: This records the type of the wicket—stumped, bowled, caught, etc.
- `BallType`: This records the type of ball played—a regular delivery, wide, no ball, or wicket.
- `RunType`: This records the type of run scored—a regular run, four, six, wide, etc.
- `PlayingPosition`: This records the playing position of a player—batsman, bowler, or all-rounder.

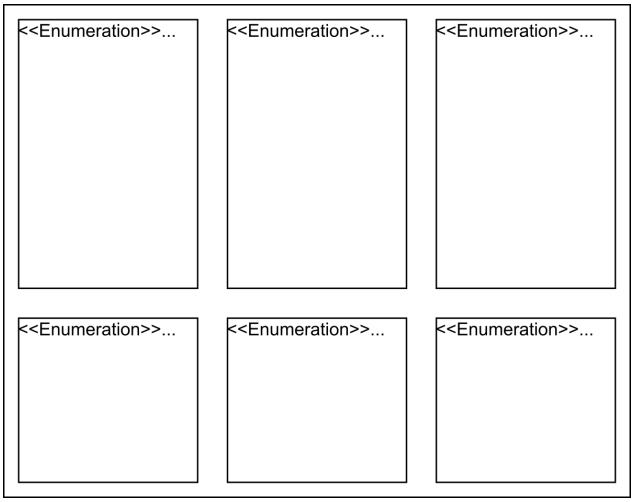


Figure 1.13: Enums in ESPNcricinfo

Custom data type

We need to create a custom data type, `Address`, that will store the physical location of any place.

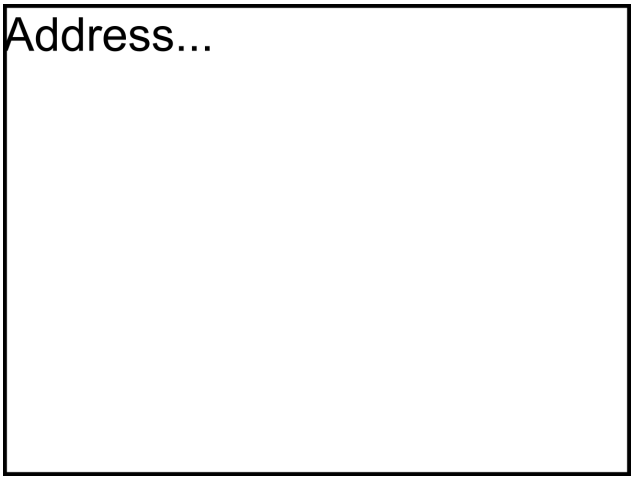


Figure 1.14: The Address custom data type

4.2 Relationship between classes

Now, we will discuss the relationships between the classes we have defined above in our ESPNcricinfo system.

Association

The class diagram has the following association relationships:

One-way association

- The `Admin` class has a one-way association with the `Player`, `Team`, `Match`, and `Tournament` classes.
- The `Player` class has a one-way association with the `Run`, `Ball`, `Wicket`, and `Over` classes.
- The `Team` class has a one-way association with the `TournamentSquad` and `Tournament` classes.
- The `TournamentSquad` class has a one-way association with the `Playing11` class.

Two-way association

- The `Ball` class is associated with the `Run`, `Wicket`, and `Commentary` classes.
- The `Team` class is associated with the `Coach` and `News` classes.
- The `Commentary` class is associated with the `Commentator` class.
- The `Match` class is associated with the `Umpire`, `Commentator`, `News`, and `Stadium` classes.
- The `Player` class is associated with the `News` classes.

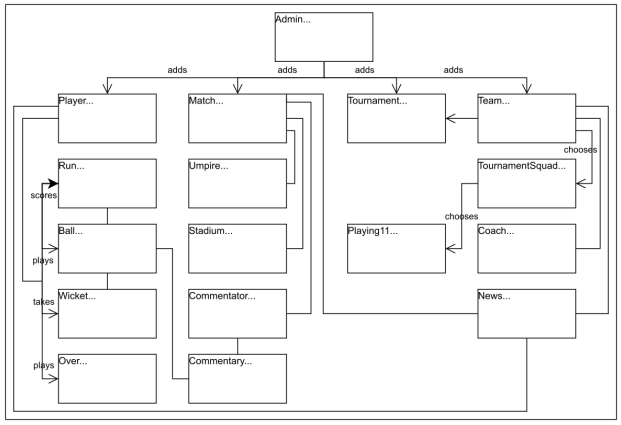


Figure 1.15: The association relationships between classes

Aggregation

The class diagram has the following aggregation relationships:

- The `Tournament` class contains the `TournamentSquad` class.

Composition

The class diagram has the following composition relationships:

- The `Player` class is composed of the `PlayerStat` class.
- The `Team` class is composed of the `Player` and `TeamStat` classes.

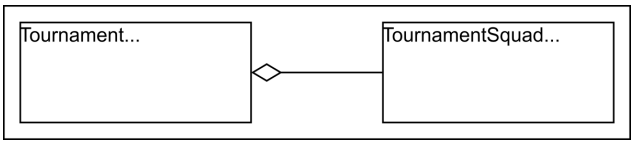


Figure 1.16: The aggregation relationship between classes

- The `Tournament` class is composed of the `Match` and `PointsTable` classes.
- The `Match` class is composed of the `Playing11`, `Innings`, and `MatchStat` classes.
- The `Innings` class is composed of the `Over` class.
- The `Over` class is composed of the `Ball` class.

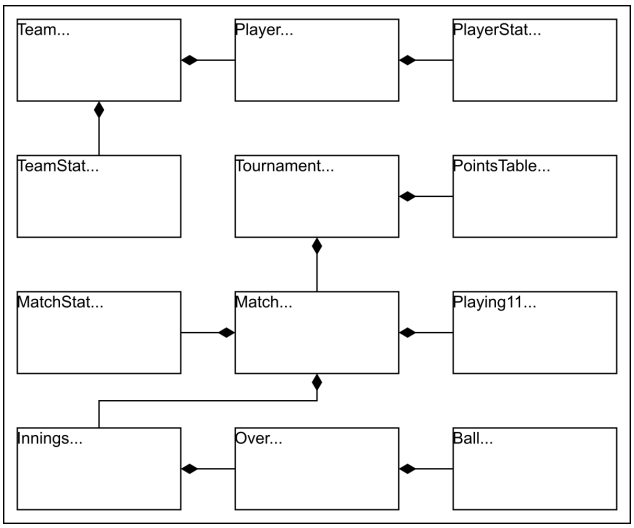


Figure 1.17: The composition relationships between classes

Inheritance

The class diagram has the following inheritance relationships:

- The `ODI`, `Test`, and `T20` classes are derived from the `Match` class.
- The `TeamStat`, `MatchStat`, and `PlayerStat` classes are derived from the `Stat` class.

  Note: We have already discussed the inheritance relationship between classes in the component section above one by one.

4.3 Class diagram of ESPNcricinfo

In this section, we outline the multiplicity (cardinality) relationships between the main classes in our ESPNcricinfo system. For each relationship, we explain the allowed number of instances on each side and the real-world or design rationale behind the connection. Understanding these relationships is key to modeling how different entities interact and collaborate to support key workflows in the system.

Source	Target	Multiplicity	Description
Player	Run	1 – 0..*	A player can score multiple runs
Player	Wicket	1 – 0..*	A player can be involved in multiple wickets
Ball	Run	1 – 0..*	A ball can result in multiple runs
Ball	Wicket	1 – 0..1	A ball can result in zero or one wicket
Ball	Player	1 – 2	Each ball has a bowler and a batsman
Over	Ball	1 – 0..*	An over can have multiple balls
Over	Player	1 – 1	One player bowls each over
Innings	Over	1 – 0..*	Each innings can have multiple overs
Innings	Playing11	1 – 2	Each innings has one batting and one bowling team
Match	Innings	1 – 0..*	Each match can have multiple innings
Match	Team	1 – 2	Each match involves two teams
Match	Stadium	1 – 1	Each match is played in one stadium
Match	Umpire	1 – 2..4	Each match can have 2 to 4 umpires
Match	Commentator	1 – 0..*	Each match can have multiple commentators
Match	MatchStat	1 – 1	Each match has one match statistics record
Team	Player	1 – 0..*	A team can have multiple players
Team	Coach	1 – 1	Each team has one coach
Team	News	1 – 0..*	A team can have multiple news entries
Team	TeamStat	1 – 1	Each team has one statistics record
Tournament	Match	1 – 0..*	A tournament includes multiple matches
Tournament	TournamentSquad	1 – 0..*	A tournament has multiple squads
TournamentSquad	Player	1 – 0..*	Each tournament squad has multiple players
TournamentSquad	Tournament	0..* – 1	Each tournament squad belongs to one tournament
Playing11	Player	1 – 0..*	A playing eleven contains multiple players
Commentary	Commentator	1 – 1	One commentator gives each commentary
News	Team	1 – 1	Each news item is linked to one team
Admin	Player	1 – 0..*	Admin manages multiple players
Admin	Team	1 – 0..*	Admin manages multiple teams

Admin	Coach	1 – 0..*	Admin manages multiple coaches
Admin	News	1 – 0..*	Admin manages multiple news entries
Admin	Tournament	1 – 0..*	Admin manages multiple tournaments
Admin	Match	1 – 0..*	Admin manages multiple matches
Admin	Stadium	1 – 0..*	Admin manages multiple stadiums
Admin	Umpire	1 – 0..*	Admin manages multiple umpires
Admin	Commentator	1 – 0..*	Admin manages multiple commentators

Here's the complete class diagram for ESPNcricinfo:

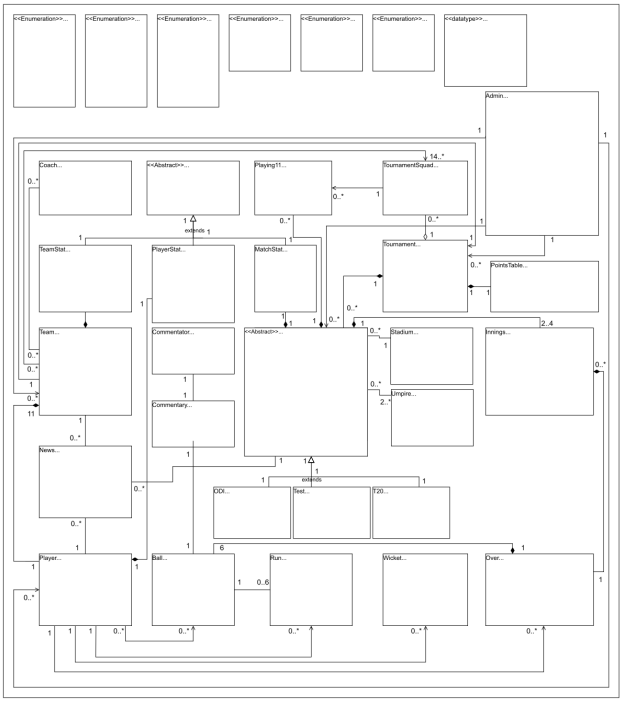


Figure 1.18: The class diagram of ESPNcricinfo

4.4 Design pattern

To implement ESPNcricinfo's core features in a flexible and scalable way, we apply the object-oriented design patterns based on system behavior.

- We know that updates, such as match scores, wickets, or player milestones, can trigger real-time commentary or notifications. To model this behavior, we can use the Observer design pattern.
- We recognize that players have different roles, such as batter, bowler, or all-rounder, and these roles can behave differently depending on the match conditions. To handle this flexible behavior, we can use the Strategy design pattern.

- We know that different match types, such as ODI, Test, and T20, need to be created dynamically based on the tournament type. To encapsulate the creation logic for these match objects, we can use the Factory design pattern.

- We know that a match consists of innings, overs, and balls in a hierarchical structure. To treat this composition uniformly, we can use the Composite design pattern.

- We know that ODI, Test, and T20 formats follow similar match structures but with variations in rules and flow. To define a common template with customizable steps, we can use the Template Method design pattern.

- We understand that the system should have a centralized administrator managing players, teams, tournaments, and matches. To enforce this single point of control, we can use the Singleton design pattern.

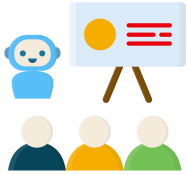
- We know that creating a match or tournament involves assembling several key components, including teams, stadiums, umpires, and statistics. To manage this step-by-step construction, we can use the Builder design pattern.

- We know that actions such as assigning umpires, adding news, or creating squads are user-triggered operations. To encapsulate these actions as separate command objects, we can use the Command design pattern.

- We recognize that commentary may need to be supplemented with live statistics, translations, or highlights to enhance the viewing experience. To add these features dynamically without altering the base class, we can use the Decorator design pattern.

4.5 AI-powered trainer

At this stage, everything should be clear. If you encounter any confusion or ambiguity, feel free to utilize the interactive AI-enabled widget below to seek clarification. This tool is designed to help you strengthen your understanding of the concepts.



We have completed the class diagram of ESPNcricinfo according to the requirements. Now , let's design the sequence diagram of ESPNcricinfo in the next lesson.

5 Sequence Diagram for ESPNcricinfo

Sequence diagrams are an effective way to visualize the interactions between different entities and objects within a system. There can be different sequence diagrams that we can create for the ESPNcricinfo system. In this lesson, we will create a sequence diagram for the following interaction:

- **Add a match:** The admin adds a new match to the system.

5.1 Add a match

The sequence diagram for adding a match should have the following actors and objects that will interact with each other:

- **Actors:** `Admin`, `Umpire`, and `Commentator`
- **Objects:** `Match` and `Stadium`

The steps involved in adding a match are listed below:

1. The admin creates a new match of a specific match type.
2. The admin adds the playing teams for the match.
3. The admin assigns a stadium to the match.
4. The admin assigns an umpire to the match.
5. The admin assigns a commentator to the match.

Based on the order above, the sequence diagram for adding a match in the ESPNcricinfo system is provided below:

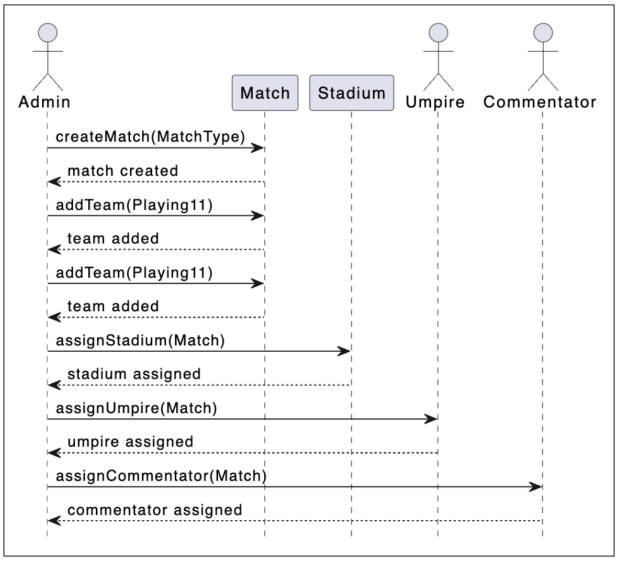


Figure 1.19: The sequence diagram for adding a match

Next, let's look at the activity diagrams for ESPNcricinfo to understand the system's control flow.

6 Activity Diagram for ESPNcricinfo

Activity diagrams are an effective way to visualize the flow of messages between activities within a system. We can create different activity diagrams for ESPNcricinfo. In this lesson, we will create an activity diagram to record the movement of a ball.

6.1 Make a record of a ball

The states and actions involved in this activity diagram are provided below.

States

Initial state: The system adds a ball to the over.
Final state: The ball record is saved.

Actions

The system adds a ball to the over, choosing the ball type that was bowled. It determines if the batter got out. If they did not, it adds to the score that was made. Finally , the commentary is added to the ball, and the ball record is saved.

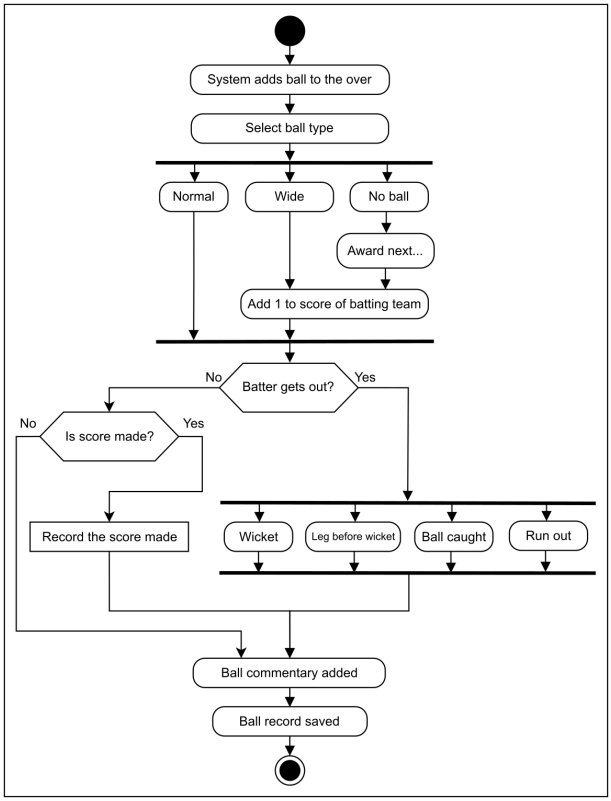


Figure 1.20: The activity diagram for making a record of a ball

In the next lesson, we will present the code for our designed classes in some of the most popular languages.

7 Code for ESPNcricinfo

We've reviewed the various aspects of ESPNcricinfo and examined the attributes associated with the problem using different UML diagrams. Let 's now explore the more practical side of things, where we will work on implementing the ESPNcricinfo problem using multiple languages. This is usually the last step in an object-oriented design interview process.
We have chosen the following languages to write the skeleton code of the different classes present in ESPNcricinfo:

- Java

- C#
- Python
- C++
- JavaScript

If you want to skip directly to the implementation and see the complete workflow, simply scroll down or click **Executable Code: Designing EspnCricinfo**.

7.1 ESPNcricinfo classes

This section will provide the skeleton code of the classes designed in the class diagram lesson.

> **Note:** For simplicity, we are not defining getter and setter functions. The reader can assume that all class attributes are private, accessed through their respective public getter methods, and modified only through their public method functions.

Constants

The following code defines the various enums and custom data types used in the ESPNcricinfo design:

> **Note:** JavaScript does not support enumerations. We'll use the `Object.freeze()` method as an alternative. It freezes an object, preventing further modifications.

```
public class Address
{
    private int zipCode;
    private string streetAddress;
    private string city;
    private string state;
    private string country;
}

public enum MatchResult
{
    LIVE,
    BAT_FIRST_WIN,
    FIELD_FIRST_WIN,
    DRAW,
    CANCELED
}

public enum UmpireType
{
    FIELD,
    RESERVED,
    THIRD_UMPIRE
}

public enum WicketType
{
    BOLD,
    CAUGHT,
    STUMPED,
    RUN_OUT,
    LBW,
    RETIRED_HURT,
    HIT_WICKET,
    OBSTRUCTING,
    HANDLING
}
```

```
public enum BallType
{
    NORMAL,
    WIDE,
    NO_BALL,
    WICKET
}

public enum RunType
{
    NORMAL,
    FOUR,
    SIX,
    LEG_BYE,
    BYE,
    NO_BALL,
    OVERTHROW
}

public enum PlayingPosition
{
    BATTING,
    BOULING,
    ALL_ROUNDER
}
```

Admin, player, coach, and umpire

The definitions of the `Admin`, `Player`, `Coach`, and `Umpire` classes are as follows:

The Admin, Player, Coach, and Umpire classes

```
public class Admin {
  private List<Player> players = new ArrayList<>();
  private List<Team> teams = new ArrayList<>();
  private List<Match> matches = new ArrayList<>();
  private List<Tournament> tournaments = new ArrayList<>();
  private List<Stat> stats = new ArrayList<>();
  private List<News> newsList = new ArrayList<>();

  public boolean addPlayer(Player player);
  public boolean addTeam(Team team);
  public boolean addMatch(Match match);
  public boolean addTournament(Tournament tournament);
  public boolean addStats(Stat stats);
  public boolean addNews(News news);

  public boolean assignStadiumToMatch(Stadium stadium, Match match);
  public boolean assignUmpireToMatch(Umpire umpire, Match match);
  public boolean assignCommentatorToMatch(Commentator commentator, Match match);
}

public class Player {
  private String name;
  private int age;
  private int country;
}

public class Coach {
  private String name;
  private int age;
  private int country;
  private List<Team> teams;
}

public class Umpire {
  private String name;
```

```
  private int age;
  private int country;

  public boolean assignMatch(Match match);
}
```

Run, ball, wicket, over, and innings

In cricket, the fundamental concepts can be categorized into five types: run, ball, wicket, over, and innings. To store information about these identities, we defined the `Run`, `Ball`, `Wicket`, `Over`, and `Innings` classes as shown below:

Implementation of the Run, Ball, Wicket, Over, and Innings classes

```
public class Run {
  private int totalRuns;
  private RunType type;
  private Player scoredBy;
}

public class Ball {
  private Player balledBy;
  private Player playedBy;
  private BallType type;
  private List<Run> runs;
  private Wicket wicket;

  public boolean addCommentary(Commentary commentary);
}

public class Wicket {
  private WicketType type;
  private Player playerOut;
  private Player balledBy;
  private Player caughtBy;
  private Player runoutBy;
  private Player stumpedBy;
}

public class Over {
  private int number;
  private Player bowler;
  private int totalScore;
  private List<Ball> balls;

  public boolean addBall(Ball ball);
}

public class Innings {
  private Playing11 bowling;
  private Playing11 batting;
  private Date startTime;
  private Date endTime;
  private int totalScores;
  private int totalWickets;
  private List<Over> overs;

  public boolean addOver(Over over);
}
```

Match

The `Match` is an abstract class representing matches in ESPNcricinfo. Matches can be of three types: `T20`, `Test`, and `ODI`. The implementation of these classes is given below:

Match and its derived classes

```
public abstract class Match {
  private Date startTime;
  private MatchResult result;
  private int totalOvers;
  private List<Playing11> teams;
  private List<Innings> innings;
  private Playing11 tossWin;
  private Map<Umpire, UmpireType> umpires;
  private Stadium stadium;
  private List<Commentator> commentators;
  private List<MatchStat> stats;

  public abstract boolean assignStadium(Stadium stadium);
  public abstract boolean assignUmpire(Umpire umpire);
  public boolean addTeam(Playing11 team);
  public boolean assignCommentator(Commentator commentator);
}

public class T20 extends Match {
  public boolean assignStadium(Stadium stadium);
  public boolean assignUmpire(Umpire umpire);
}

public class Test extends Match {
  public boolean assignStadium(Stadium stadium);
  public boolean assignUmpire(Umpire umpire);
}

public class ODI extends Match {
  public boolean assignStadium(Stadium stadium);
  public boolean assignUmpire(Umpire umpire);
}
```

Team, tournament squad, and playing11

The definitions of the `AdminTeam`, `TournamentSquad`, and `Playing11` classes are as follows:

The Team, TournamentSquad, and Playing11 classes

```
public class Team {
  private String name;
  private List<Player> players;
  private Coach coach;
  private List<News> news;
  private TeamStat stats;

  public boolean addSquad(TournamentSquad squad);
  public boolean addPlayer(Player player);
  public boolean addNews(News news);
}

public class TournamentSquad {
  private List<Player> players;
  private Tournament tournament;
  private List<TournamentStat> stats;

  public boolean addPlayer(Player player);
}
```

```
public class Playing11 {
  private List<Player> players;

  public boolean addPlayer(Player player);
}
```

Tournament, points table, and stadium

The definitions of the `Tournament`, `PointsTable`, and `Stadium` classes are as follows:

The Tournament, PointsTable, and Stadium classes

```
public class Tournament {
  private Date startDate;
  private List<TournamentSquad> teams;
  private List<Match> matches;
  private PointsTable points;

  public boolean addTeam(TournamentSquad team);
  public boolean addMatch(Match match);
}

public class PointsTable {
  private HashMap<String, float> teamPoints;
  private HashMap<Team, MatchResult> matchResults;
  private Tournament tournament;
  private Date lastUpdated;
}

public class Stadium {
  private String name;
  private Address location;
  private int maxCapacity;

  public boolean assignMatch(Match match);
}
```

Commentator, commentary, and news

The definitions of the `Commentator`, `Commentary`, and `News` classes are as follows:

Commentator, Commentary, and News classes

```
public class Commentator {
  private String name;

  public boolean assignMatch(Match match);
}

public class Commentary {
  private String text;
  private Date createdAt;
  private Commentator commentator;
}

public class News {
  private Date date;
  private String text;
  private List<byte> image;
  private Team team;
}
```

Statistics

The `Stat` is an abstract class that represents the statistics in ESPNcricinfo. Statistics can be of three types: `PlayerStat`, `MatchStat`, and `TeamStat`. These classes are represented below:

Stat and its derived classes

```
public abstract class Stat {
  public abstract boolean updateStats();
}

public class PlayerStat extends Stat {
  private int ranking;
  private int bestScore;
  private int bestWicketCount;
  private int totalMatchesPlayed;
  private int total100s;
  private int totalHattricks;

  public boolean updateStats();
}

public class MatchStat extends Stat {
  private double winPercentage;
  private Player topBatsman;
  private Player topBowler;

  public boolean updateStats();
}

public class TeamStat extends Stat {
  private int totalSixes;
  private int totalFours;
  private int totalReviews;

  public boolean updateStats();
}
```

7.2 Executable code: Designing ESPNcricinfo

Below is a fully self-contained, runnable program in Java, C#, C++, Python, and JavaScript that demonstrates the core workflows of the ESPNcricinfo system. The main driver code of the system resides in the `Driver.java`, `Driver.cs`, `Driver.py`, `Driver.cpp`, and `Driver.js` for all respective languages. You can click the "Run" button to execute the codes.

What does this code show

- **System initialization:** Initializes the system by creating a new tournament with an associated PointsTable and Admin instance. The tournament is configured with a start date, and its internal lists for teams and matches are prepared. This forms the foundational setup for all cricket-related scenarios.

- **Scenario 1 (Match creation and assignment):** An admin creates a T20 match and sets the start time and number of overs. A stadium named "Wankhede Stadium" is assigned to the match along with key roles: an umpire ("Aleem Dar") and a commentator ("Harsha Bhogle"). This scenario demonstrates how a match is structured and populated with essential entities.

- **Scenario 2 (Team creation and player assignment):** Two players, Virat Kohli and Mitchell Starc, are created and added to two separate teams using Playing11. These teams are then registered to the match and the tournament as TournamentSquads. This shows how teams and players are formed and tied to both the match and the broader tournament.

- **Scenario 3 (Ball delivery and commentary):** A ball is simulated where Starc bowls to Kohli, who scores a boundary (4 runs). The ball is linked to a Run object, and a Commentary entry is recorded, highlighting Kohli's stroke. This scenario reflects real-time in-game activity and how gameplay details and commentary are captured.
- **Scenario 4 (Points table update):** The points table is updated after the simulated match: Team 1 earns 2 points, while Team 2 earns none. The latest standings are printed, along with the time stamp of the most recent update. This scenario shows how team performance affects tournament rankings.

  Hint: Try customizing the code!
  This code is designed not just for reading, but for experimentation. You can **edit, extend, or modify** any part of the example.

```java
import java.util.*;

public class Driver {
    public static void main(String[] args) {
        System.out.println(" === Welcome to Cricinfo System Simulation ===
");

        // ==================================================
        //  Initialization: Tournament and Admin Setup
        // ==================================================
        System.out.println(" === Initialization ===
");

        Tournament tournament = new Tournament();
        tournament.setStartDate(new Date());
        tournament.setTeams(new ArrayList<>());
        tournament.setMatches(new ArrayList<>());

        PointsTable pointsTable = new PointsTable();
        pointsTable.setTeamPoints(new HashMap<>());
        pointsTable.setMatchResults(new HashMap<>());
        pointsTable.setTournament(tournament);
        pointsTable.setLastUpdated(new Date());
        tournament.setPoints(pointsTable);

        Admin admin = new Admin();

        System.out.println(" Tournament created and points table initialized.");
        System.out.println("  Tournament Start Date: " + tournament.getStartDate());
        System.out.println("  Points table last updated: " + pointsTable.getLastUpdated() + "
");

        // ==================================================
        //  Scenario 1: Match Creation and Assignment
        // ==================================================
        System.out.println(" === Scenario 1: Create Match and Assign Roles ===
");

        Match match = admin.createMatch(MatchType.T20);
        match.setStartTime(new Date());
        match.setTotalOvers(20);
        System.out.println(" Match Created: T20 Format");
        System.out.println("  Match Start Time: " + match.getStartTime());
        System.out.println("  Total Overs: " + match.getTotalOvers());

        Stadium stadium = new Stadium();
        stadium.setName("Wankhede Stadium");
        Address address = new Address();
        address.setCity("Mumbai");
        address.setCountry("India");
        stadium.setLocation(address);
        admin.assignStadiumToMatch(stadium, match);
        System.out.println(" Stadium Assigned: " + stadium.getName() + ", " + address.getCity() + ",
             " + address.getCountry());
```

```
        Umpire umpire = new Umpire();
        umpire.setName("Aleem Dar");
        umpire.setAge(50);
        umpire.setCountry(92);
        admin.assignUmpireToMatch(umpire, match);
        System.out.println(" Umpire Assigned: " + umpire.getName());

        Commentator commentator = new Commentator();
        commentator.setName("Harsha Bhogle");
        admin.assignCommentatorToMatch(commentator, match);
        System.out.println(" Commentator Assigned: " + commentator.getName());

        tournament.addMatch(match);
        System.out.println(" Match added to tournament.
");

        // ==================================================
        //  Scenario 2: Add Players and Form Teams
        // ==================================================
        System.out.println(" === Scenario 2: Add Players and Teams ===
");

        Player player1 = new Player();
        player1.setName("Virat Kohli");
        player1.setAge(35);
        player1.setCountry(91);

        Player player2 = new Player();
        player2.setName("Mitchell Starc");
        player2.setAge(34);
        player2.setCountry(61);

        Playing11 team1 = new Playing11();
        team1.addPlayer(player1);

        Playing11 team2 = new Playing11();
        team2.addPlayer(player2);

        match.addTeam(team1);
        match.addTeam(team2);
        System.out.println(" Players added to match:");
        System.out.println("  Team 1: " + player1.getName());
        System.out.println("  Team 2: " + player2.getName());

        TournamentSquad squad1 = new TournamentSquad();
        squad1.setPlayers(List.of(player1));
        tournament.addTeam(squad1);

        TournamentSquad squad2 = new TournamentSquad();
        squad2.setPlayers(List.of(player2));
        tournament.addTeam(squad2);
        System.out.println(" Teams registered to the tournament.
");

        // ==================================================
        //  Scenario 3: Simulate Gameplay and Commentary
        // ==================================================
        System.out.println(" === Scenario 3: Ball Played and Commentary ===
");

        Ball ball = new Ball();
        ball.setBalledBy(player2);
        ball.setPlayedBy(player1);
        ball.setType(BallType.NORMAL);

        Run run = new Run();
        run.setTotalRuns(4);
        run.setType(RunType.FOUR);
        run.setScoredBy(player1);
        ball.setRuns(List.of(run));
```

```
        Commentary commentary = new Commentary();
        commentary.setText("Kohli punches it through the covers for a boundary!");
        commentary.setCreatedAt(new Date());
        commentary.setCommentator(commentator);
        ball.addCommentary(commentary);

        System.out.println(" Ball Delivered:");
        System.out.println("  Bowler: " + player2.getName());
        System.out.println("  Batsman: " + player1.getName());
        System.out.println("  Runs Scored: " + run.getTotalRuns() + " (" + run.getType() + ")");
        System.out.println("  Commentary: "" + commentary.getText() + "" - " + commentator.getName()
             + "
");

        // ================================================
        //  Scenario 4: Update and Display Points Table
        // ================================================
        System.out.println(" === Scenario 4: Points Table Update ===
");

        pointsTable.getTeamPoints().put("Team 1", 4.0f);
        pointsTable.getTeamPoints().put("Team 2", 0.0f);
        pointsTable.setLastUpdated(new Date());

        System.out.println(" Updated Points Table:");
        for (Map.Entry<String, Float> entry : pointsTable.getTeamPoints().entrySet()) {
            System.out.println("  " + entry.getKey() + ": " + entry.getValue() + " points");
        }

        System.out.println("
 Points Table Last Updated On: " + pointsTable.getLastUpdated());
        System.out.println("
 === Cricinfo Tournament Simulation Complete ===");
    }
}
```

7.3 Wrapping up

We've explored the complete design of ESPNcricinfo in this chapter. We 've looked at how ESPNcricinfo can be visualized using various UML diagrams and designed using object-oriented principles and design patterns.

8 Summary: Designing ESPNcricinfo

With the ESPNcricinfo case study complete, it's time to review and strengthen your understanding. We 'll revisit the primary system requirements, break down the core classes and their roles and relationships, and highlight the essential design principles. Additionally , we'll explore how objects interact within the system and walk through the full workflow to see how all the components integrate to deliver the required features.

8.1 Key requirements

Here are the primary functional and non-functional requirements that define the ESPNcricinfo system:

1. Admins can add or update player, team, and match statistics.
2. Commentators can manage ball-by-ball commentary and scoring events.
3. The admin can track and update all match types (Test, ODI, T20).
4. The system maintains records and points tables for all tournaments, current and past.

5. Results for all previous matches are available based on stored match data.
6. Coaches can create and submit squads for tournaments.
7. The system enables the selection of the playing eleven from the squad for each match.
8. Admins can manage all aspects of the system, including entities like teams, stadiums, and news.

8.2 System actors

The following actors interact with the ESPNcricinfo system:

- **Admin:** Manages all entities in the system, including tournaments, teams, players, matches, stats, and news.
- **Coach:** Manages team players, submits squads for tournaments, and adds team news.
- **Umpire:** Assigned to matches, oversees decisions, and maintains game fairness.
- **Commentator:** Adds or edits ball-by-ball commentary for live matches.

8.3 Important classes and relationships

Here's an overview of the core classes, their responsibilities, and their main relationships:

Class	Description	Important Relationships
`Admin`	Manages the creation and modification of all major entities.	Associates with `Player`, `Team`, `Match`, `Tournament`
`Team`	Represents a cricket team, including players, coach, news, and stats.	Composed of `Player`, `TeamStat`; associated with `Coach`, `News`
`Player`	Represents an individual cricketer and their statistics.	Composed of `PlayerStat`; associated with `Run`, `Ball`, `Wicket`, `Over`
`Coach`	Manages players and team news; submits squads.	Associated with `Team`, `News`
`Umpire`	Oversees matches assigned by the admin.	Associated with `Match`
`Commentary`	Records commentary for each ball of an over.	Associated with `Commentator`, `Ball`
`Match`	Base class for different match types (Test, ODI, T20), managing match information and statistics.	Has child classes: Test, ODI, T20; composed of Playing11, Innings, MatchStat; associated with Umpire, Commentator, Stadium
`Tournament`	Represents a tournament, containing matches and a points table.	Composed of `Match`, `PointsTable`, and contains `TournamentSquad`
`TournamentSquad`	A squad selected by a team for a tournament.	Aggregated in `Tournament`; associated with `Playing11`
`Ball`	Records each ball bowled, runs, wickets, and commentary.	Associated with `Run`, `Wicket`, `Commentary`; part of `Over`
`Innings`	Represents one team's innings in a match.	Composed of `Over`; part of `Match`
`PointsTable`	Shows points and rankings for a tournament.	Part of `Tournament`

8.4 Design highlights

Here are some notable features and choices in the ESPNcricinfo system:

8.5 Design approach

The system is designed using a bottom-up approach, starting from the smallest units, such as `Ball`, `Run`, and `Wicket`, and then assembling them into higher-level entities, including `Over`, `Innings`, `Match`, and ultimately the full ESPNcricinfo platform.

8.6 Design principles

The following SOLID principles are reflected in the system design:

- **Single Responsibility principle (SRP):** Each class is focused on a single responsibility, such as `Player` for cricketer details, `Ball` for per-delivery events, and `Commentary` for match commentary.
- **Open/Closed principle (OCP):** New match types or statistics can be added by extending base classes (e.g., new child classes of `Match` or `Stat`).
- **Liskov Substitution principle (LSP):** Subclasses such as `ODI`, `Test`, and `T20` can be used wherever the base `Match` is expected.
- **Interface Segregation principle (ISP):** Responsibilities, such as adding commentary or assigning umpires, are split among appropriate classes and actors.
- **Dependency Inversion principle (DIP):** High-level modules (`Admin`, `Coach`) depend on abstractions, such as manipulating generic `Match` or `Team` objects, not concrete implementations.

8.7 Design patterns

The design pattern below is used in the ESPNcricinfo system:

- **Factory pattern:** Used for dynamic creation of different match types, tournaments, and squads at runtime without exposing concrete classes.

8.8 Object interactions

Key interactions between objects include:

- **Adding a match:** The admin creates a new match (Test, ODI, or T20), adds teams and a stadium, and assigns umpires and commentators. Each action updates the related objects and relationships within the system.
- **Recording a ball:** For each ball bowled, the system records its outcome (runs, wickets, commentary) by updating Ball, Run, Wicket, and Commentary objects within the current Over and Innings.
- **Squad and playing 11 management:** Coaches manage `TournamentSquads` and choose the `Playing11` for each match, maintaining links between `Team`, `TournamentSquad`, and `Playing11` objects.

8.9 System workflow

These are the major user journeys and life cycle events in ESPNcricinfo:

- **Tournament setup:** Admin adds tournaments and teams; coaches submit their squads.
- **Live match execution:** Ball-by-ball events (deliveries, runs, wickets, commentary) are entered and stats are update in real time.

- **Post-match processing:** Match outcomes are recorded, the points table is updated, and news or analysis can be added.
- **Statistics and history:** Users and admins can view stats and commentary for any player, team, match, or tournament, including historical data.

9 Key Insights and Tips: Designing ESPNcricinfo

Well done on completing the ESPNcricinfo case study! In this lesson, we'll take your understanding further by exploring common design mistakes and sharing practical interview strategies. You 'll also have a chance to test your knowledge with a short quiz and find related case studies to help you reinforce and expand your object-oriented design expertise.

9.1 Common mistakes

Avoid these common mistakes to design a robust and scalable ESPNcricinfo-like system:

- **Poor modularization of entities:** Mixing details for matches, players, teams, and commentary in a single class leads to complex, unmanageable code. Use independent classes for each domain (e.g., `Match`, `Player`, `Ball`, `Commentary`).
- **Ignoring real-time updates:** Failing to handle real-time match updates (e.g., ball-by-ball scores, commentary) results in stale or laggy user experiences. Use appropriate patterns (like Observer) for live notifications and UI refreshes.
- **Inadequate statistics modeling:** Failing to design extensible classes for statistics (such as `PlayerStat`, `TeamStat`, and `MatchStat`) makes it difficult to add new types of statistics or aggregate data across matches and tournaments.
- **Unclear role and permission management:** Confusing the roles of admin, coach, umpire, and commentator can result in unauthorized data changes or missed responsibilities. Make actor capabilities explicit.
- **Incomplete handling of match types:** Failing to distinguish between match types (Test, ODI, T20) results in feature gaps. Use class inheritance for match type-specific logic.

9.2 Interview tips

Understand key design patterns, system components, and real-world applications to prepare for your interview and confidently discuss your approach to these concepts.

Tip	**Explanation**
Show system modularity with clear class boundaries.	Explain why you're designing independent classes for domains like Match, Player, Commentary, and Stat, and describe their relationships (e.g., aggregation, composition).
Map real system events to OOD artifacts.	For every real-world cricket event (wicket, over, run), map the system event to the responsible class/object and describe the update flow.
Discuss extensibility.	Show how your design accommodates new match types, stats, or user roles in the future.
Talk about real-time updates.	Discuss patterns (e.g., Observer) that enable live score and commentary updates.
Clarify responsibilities.	Explain why only Admins can add matches or tournaments, and why only Commentators can add live commentary.

9.3 Test your understanding

Now that you understand OOD principles well, solve the quiz below to test your knowledge.

Designing ESPNcricinfo

Question 1: Which class relationship best models the idea that an over consists of multiple `Ball` objects?

- Inheritance
- **[CORRECT]** Aggregation
  Explanation: Aggregation models the relationship where an Over consists of multiple `Ball` objects, grouping them together as parts of the whole. Each Ball still exists independently, but together they form the structure of an Over.
- Association
- Dependency Injection

Question 2: Who is responsible for adding live commentary during a match?

- Admin
- Umpire
- **[CORRECT]** Commentator
  Explanation: Only the commentator role is responsible for adding or updating live commentary during a match, ensuring that commentary is managed by a dedicated expert and keeping responsibilities clear within the ESPNcricinfo system.
- Coach

Question 3: Which design pattern is most appropriate for updating all viewers when a new ball is bowled?

- Factory pattern
- Singleton pattern
- **[CORRECT]** Observer pattern
  Explanation: Observer allows the system to notify all registered views or listeners (like user devices or dashboards) of live updates instantly.
- Builder pattern

Question 4: Why does the system model `Innings` as a separate class from `Match`?

- To avoid inheritance
- **[CORRECT]** Because a single match can have multiple innings, each with unique details
  Explanation: Modeling `Innings` separately enables the system to accurately represent matches (like Test and ODI) with multiple, distinct innings.

- To make the class diagram more complex
- Because only test matches need innings

Question 5: What OOD principle is demonstrated by having `Test`, `ODI`, and `T20` all extend from the base `Match` class?

- Encapsulation
- **[CORRECT]** Inheritance
  Explanation: Inheritance allows each match type to share common structure and behavior while enabling specific rule overrides.
- Association
- Composition

9.4 Related case studies

The following case studies will help reinforce concepts used in the ESPNcricinfo:

- **Designing Facebook:** Explores designing scalable news feeds, notifications, user profiles, comments, and real-time interactions—concepts directly applicable to ESPNcricinfo's event feeds and user engagement.
- **Designing LinkedIn:** Teaches designing complex profile management, connections, activity streams, and notifications, similar to how ESPNcricinfo handles player/team profiles and user following systems.
- **Design of a sports statistics platform:** Covers how to model and manage player, team, and match stats across multiple sports.
- **Design of a live event notification system:** Focuses on real-time updates and notification triggers for ongoing events (e.g., sports, concerts).
- **Design of a tournament management system:** Shows modular design for teams, brackets, stats, and live updates in multi-team competitions.